

Test Project Outline – Module G – SwapLoop Automated Testing

Competition time

Competitors will have **3 hours** to complete this module.

Introduction

SwapLoop is a fictional Shanghai community pilot for safer e-bike battery handling: riders swap removable batteries or use monitored charging bays at stations, while delivery partners pay via subscription plans with usage-based overage.

This module is **testing-only**. Two working applications are provided:

1. **Frontend testing** — a SwapLoop **multi-page rider UI** (Module D behaviour) talking to a Module C-compatible API, with **Cypress** test skeletons.
2. **Backend testing** — a **partner billing service** (Module B billing ideas) implemented in PHP, with **PHPUnit** test skeletons.

Competitors **must not change** application source. Fill in the empty test bodies so the suites verify the behaviours described below.

Workshop tooling and Docker layout follow [ws26-cypress](#) and [ws26-phpunit](#).

General Description of Project and Tasks

What you receive

Submodule	Path	Your work
Rider UI + Cypress	<code>assets/frontend-testing/</code>	Implement specs under <code>cypress/e2e/</code>
Billing + PHPUnit	<code>assets/backend-testing/</code>	Implement tests under <code>tests/</code>

Rules

- Do **not** edit `public/`, `server.js`, or PHP under `src/` (except if a handout explicitly allows a config file).
- Prefer stable selectors already present on the pages (`data-testid`, documented ids). Do not rely on brittle CSS layout classes alone when a test id exists.
- Use mocks, stubs, and intercepts where the scenario requires isolation from real I/O or HTTP.

- How you assert is up to you; marking checks that each listed expectation is covered correctly.

Suggested time split

Block	Focus	Approx. time
A	Cypress rider flows	~1.5–2 h
B	PHPUnit billing	~1–1.5 h

Seed credentials (frontend)

Email	Password	Notes
lin.xiaoyu@swaploop.test	password123	Active swappable rider (SL-48)
chen.wei@swaploop.test	password123	Active integrated rider (GB-AC-48)
sun.hao@swaploop.test	password123	Suspended — login must fail with the API's 403 message

Physical vocabulary (frontend)

Term	Meaning
SwapLoop Station	Location (SWAP , CHARGING , or HYBRID)
Battery Slot	Swap bay (SWAP_BAY)
E-bike Charging Bay	Whole-bike bay (BIKE_BAY)

Part A — Frontend testing (Cypress)

The multi-page rider UI under `assets/frontend-testing/` is a working Module D-style client (`login.html`, `stations.html`, `station.html`, `activity.html`, `scan.html`, ...). A Module C-compatible mock API is served with the app (or assessors may attach a real Module C Main Backend). Specs already contain `describe` / `it` titles; implement each `it` body.

A1 · Login (01_login.cy.js)

Test	Expectation (what)
loads the login page	Login screen is reachable and shows email, password, and submit controls
shows an error when email is missing	Submitting without email surfaces a clear validation message; user stays on login
shows an error for invalid credentials	Wrong email/password surfaces the invalid-credentials message from the API
blocks suspended accounts	Suspended seed account cannot enter the app; suspended message is distinct from invalid credentials
signs in and reaches stations	Valid rider signs in, lands in the authenticated stations experience, and the session identity is visible

A2 · Register (02_register.cy.js)

Test	Expectation (what)
requires vehicle profile fields for swappable mode	Swappable registration without battery type does not create an account
completes two-step register for a swappable rider	After vehicle profile + simulated Alipay link step, a new swappable rider can create an account and enter the app
completes two-step register for an integrated rider	Same flow works for integrated mode with a connector type and no battery type

A3 · Stations list (03_stations.cy.js)

Test	Expectation (what)
lists stations for a signed-in rider	After login, the stations list shows seeded stations with name and type
filters by station type	Choosing a type filter restricts the list to that type (e.g. only SWAP)
shows compatible availability indication	For the signed-in rider profile, each relevant card indicates whether a compatible ready battery or bay is available
unauthenticated visitors can browse stations	Without signing in, a visitor can open the public stations browse path and see the list

A4 · Station detail and reserve (04_station_detail.cy.js)

Test	Expectation (what)
opens station detail from a list card	Navigating from a card shows that station's hub (identity, status, availability guidance)
reserves a swap hold when eligible	An eligible swappable rider can reserve a battery; the UI shows the active hold and a path to Activity
surfaces last-charge conflict without inventing availability	When reserve returns a conflict, the rider sees a clear "not available anymore" style message and reserve is not left looking successful
blocks a second reserve while another station hold is active	If the rider already has an active service elsewhere, a new reserve is blocked and Activity is reachable

A5 · Activity — swap (05_activity_swap.cy.js)

Test	Expectation (what)
shows countdown and actions while reserved	Active swap in <code>RESERVED</code> shows countdown-related UI plus start and cancel controls
starts and confirms a swap	Rider can progress reserved → started → confirmed and then see a receipt with amount from the API (not invented)
cancels a reserved swap	Cancel ends the active service and Activity reflects that it is gone

A6 · Activity — charging (06_activity_charging.cy.js)

Test	Expectation (what)
starts charging from a reserved hold	Eligible integrated rider can start charging from Activity
reflects live charging status until ready	While charging, UI reflects live status from the API until ready-for-collection
collects the bike and shows a receipt	Collect completes the journey and shows the API receipt fields

A7 · Station QR scan (07_qr_scan.cy.js)

Test	Expectation (what)
embeds the QR emulator	Scan screen exposes the QR emulator surface
navigates to station hub on a valid poster payload	Emitting a canonical station deep link opens that station's detail hub
rejects a mismatched payload	Non-station payloads show a clear mismatch message and do not open a false station

A8 · HTTP intercepts and errors (`08_http_errors.cy.js`)

Test	Expectation (what)
handles protected-route unauth by returning to login	Clearing the session and opening a protected view returns the user to login
shows an actionable message on server error during reserve	When reserve is forced to fail with a server error, the UI shows an actionable error (no stack trace)
prevents double-submit while reserve is pending	While a reserve request is in flight, the rider cannot fire a duplicate successful reserve from the same control

Part B — Backend testing (PHPUnit)

The PHP package under `assets/backend-testing/` calculates partner subscription billing (Module B ideas) **without a database** and **without Composer or third-party libraries**. `BillingService` uses PHP `date()` and `file_put_contents()`, then notifies via a concrete `Notifier` class (no interface). Domain code uses **inheritance** and a **factory**.

Study `src/`. Implement each incomplete test under flat `tests/*.php`.

Billing rules under test

1. Caller supplies a `SubscriptionPlan` and a use count (not SQL).
2. Plan: monthly **base fee**, **included quota**, **overage** at the plan rate.
3. **Volume discount tier** matches month's usage; discount applies to **overage only**.
4. Amounts are whole **CNY yuan**.
5. Export filenames include period and **today's date** from `date('Y-m-d')`.
6. `Notifier` runs only after a successful write; failed writes / invalid usage must not notify.

B1 · Billing calculator (`BillingCalculatorTest`)

Test	Expectation (what)
calculates base fee only when uses stay within quota	Within quota → total equals base fee; overage and discount are zero
calculates overage without discount when no tier matches	Uses above quota with no matching tier → overage charged at full rate
applies volume discount to overage only	Matching tier reduces overage portion only; base fee unchanged
rejects negative usage	Negative use counts are rejected

B2 · Discount tiers (`DiscountTierTest`)

Test	Expectation (what)
selects the highest matching tier for a usage count	<code>SubscriptionPlan::matchTier</code> picks the correct tier
returns no tier when usage is below all thresholds	Low usage yields no discount tier

B3 · Billing documents / inheritance (`BillingDocumentTest`)

Test	Expectation (what)
partner summary exposes expected breakdown fields	Partner, period, plan, uses, quota, overage, fees, discount, total
finance CSV renders a header and partner rows	Header row + one row per partner (id, name, period, plan, uses, total)
document types share a common abstraction	Concrete types extend <code>BillingDocument</code>

B4 · Document factory (`DocumentFactoryTest`)

Test	Expectation (what)
creates a partner summary document	Factory returns <code>PartnerSummary</code> for that kind
creates a finance CSV document	Factory returns <code>FinanceCsv</code> for that kind
rejects an unknown document kind	Unknown kind is rejected

B5 · Billing service (`BillingServiceTest`)

Test	Expectation (what)
writes a partner summary file whose name includes the current date	Real file under the output dir; basename includes <code>date('Y-m-d')</code> and partner/period
written file contains breakdown	File contents reflect the calculated billing fields
notifies after a successful write	After a real successful write, <code>Notifier</code> is invoked once
does not notify when write fails	Bad/missing output dir → error; <code>Notifier</code> must not run
does not write or notify when usage is invalid	Negative uses → no file and no notify

Assessment

Assessment runs the provided applications unchanged and evaluates the competitor's tests against the expectations above (automated where possible, expert review of coverage and assertion quality). Empty or incomplete cases score zero for that aspect.

Mark distribution

WSOS SECTION	Description	Points
1	Work organization and self-management	5
2	Communication and interpersonal skills	5
4	Front-End Development (Cypress)	45
5	Back-End Development (PHPUnit)	45
Total		100